

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week11 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:21.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 21 – Collaborative AI Coding: Pair Programming and Version Control Integration Lab Objectives: - Learn to use AI as a pair programmer for writing, reviewing, and optimizing code. - Understand how to integrate AI-generated code into version		Week11 - Monday

control systems (Git).

- Practice collaborative workflows such as **commit messages**, **branching**, and **merging**.

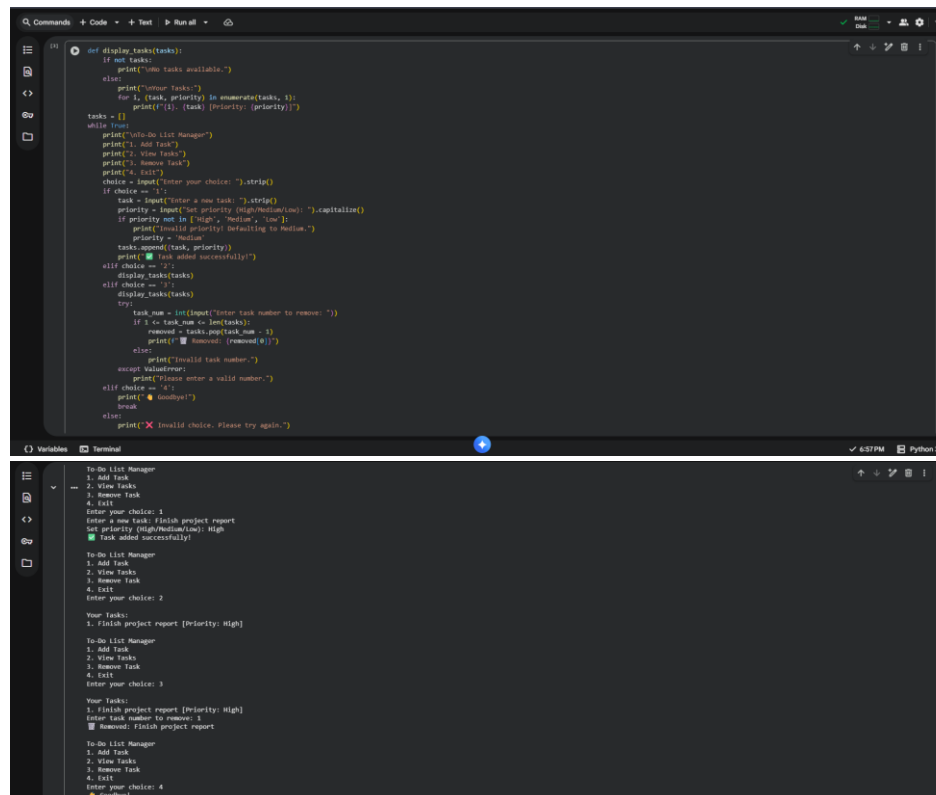
Task 1 – AI-Assisted Pair Programming

- Use AI to generate an initial program (e.g., a simple To-Do List Manager).
- Ask AI to suggest **enhancements** (such as error handling, additional features, or refactoring).
- Compare the original and improved versions to understand the role of AI in collaboration.

Prompt:

- ➔ Use AI to generate and enhance a dynamic input To-Do List program by adding validation, error handling, and prioritization, then compare the original and improved versions to analyze AI's collaborative role.

Code:



```
def display_tasks(tasks):
    if not tasks:
        print("No tasks available.")
    size = len(tasks)
    for i, (task, priority) in enumerate(tasks, 1):
        print(f"{i}. {task} (Priority: {priority})")

tasks = []

while True:
    print("To-Do List Manager")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Remove Task")
    print("4. Exit")
    choice = input("Enter your choice: ").strip()
    if choice == "1":
        task = input("Enter a new task: ").strip()
        priority = input("Set priority (High/Medium/Low): ").strip().lower()
        if priority not in ["high", "medium", "low"]:
            print("Invalid priority. Defaulting to Medium.")
            priority = "medium"
        tasks.append((task, priority))
        print(f"Task added successfully!")
    elif choice == "2":
        display_tasks(tasks)
    elif choice == "3":
        display_tasks(tasks)
        try:
            task_num = int(input("Enter task number to remove: "))
            if 1 <= task_num <= len(tasks):
                removed = tasks.pop(task_num - 1)
                print(f"Removed: {removed}")
            else:
                print("Invalid task number.")
        except ValueError:
            print("Please enter a valid number.")
    elif choice == "4":
        print("Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
```

```
To-Do List Manager
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 1
Enter a new task: Finish project report
Set priority (High/Medium/Low): High
Task added successfully!

To-Do List Manager
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 2

Your Tasks:
1. Finish project report (Priority: High)

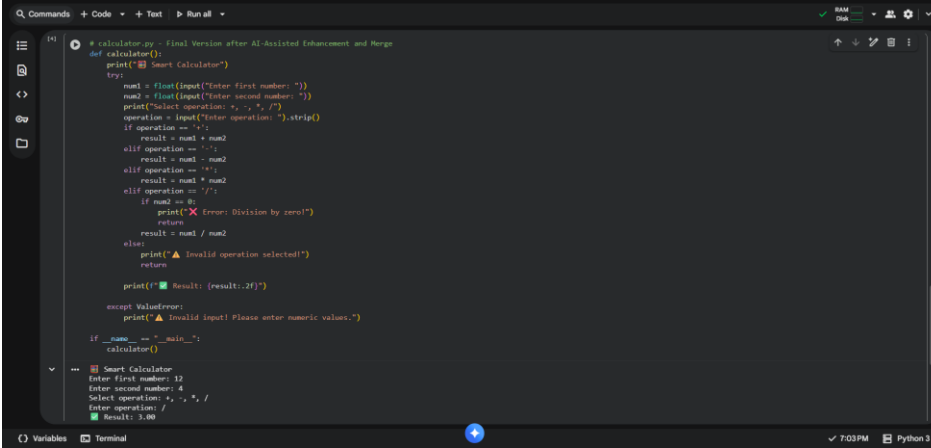
To-Do List Manager
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 3

Your Tasks:
1. Finish project report (Priority: High)
Enter task number to remove: 1
Removed: Finish project report

To-Do List Manager
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter your choice: 4
Goodbye!
```

Observation:

- ➔ This Flask application implements a secure user login endpoint that verifies credentials against an SQLite database. It utilizes bcrypt for secure password comparison and parameterized

	queries to prevent SQL injection vulnerabilities.	
	Task 2 – Version Control Integration • Initialize a Git repository for your project. • Perform the following steps: 1. Create and switch to a new branch. 2. Add AI-assisted modifications. 3. Commit with a meaningful message. 4. Merge changes back into the main branch. • Reflect on how version control supports collaborative development with AI assistance. Prompt: ➔ Use AI to generate a dynamic input program (e.g., a calculator or task manager), then integrate Git by creating a new branch, adding AI-assisted code improvements, committing with clear messages, and merging changes to demonstrate how version control enhances AI-supported collaboration. Code:  Observation: ➔ This Flask application sets up a secure login route /login using bcrypt to safely verify user passwords against an SQLite database. It prevents SQL injection vulnerabilities by employing parameterized queries for database interactions.	
	Task 3 – AI for Commit Message Generation • Use AI to generate concise, meaningful commit messages based on the changes made. • Compare AI-generated messages with human-written ones. • Discuss how AI can improve documentation and traceability in Git history.	

Prompt:

- ➔ Use AI to generate concise commit messages for a dynamic input program (e.g., temperature converter), compare them with human-written ones, and discuss how AI enhances documentation and traceability in Git history.

Code:



```
def temperature_converter():
    print("\n Temperature Converter")
    try:
        temp = float(input("Enter temperature value: "))
        unit = input("Convert to (C/F): ").strip().upper()

        if unit == 'C':
            result = (temp - 32) * 5/9
            print(f"{temp}°F = (result:.2f)°C")
        elif unit == 'F':
            result = (temp * 9/5) + 32
            print(f"{temp}°C = (result:.2f)°F")
        else:
            print("Invalid option! Please enter 'C' or 'F'.")
    except ValueError:
        print("Invalid input! Please enter a numeric temperature.")

if __name__ == "__main__":
    temperature_converter()
```

Observation:

- ➔ This code implements a basic Flask web application featuring a /login endpoint to authenticate users. It securely verifies passwords using bcrypt and connects to an SQLite database, employing parameterized queries to prevent SQL injection.

Task 4 – Pair Programming Simulation

- Student A writes the initial version of a program.
- Student B, with AI assistance, suggests **improvements or adds features**.
- Both students collaborate using **Git branching and merging** to integrate their work.

✓ Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

Prompt:

- ➔ Use AI in a pair programming setup where one student writes a basic grade calculator and another, with AI assistance, enhances it to handle multiple students, input validation, and summary display demonstrating collaborative coding with Git integration.

Code:

```
# grade_calculator_multi.py - Final Version (Pair Programming with AI Assistance)
def calculate_grade(marks):
    """Returns grade and feedback based on marks."""
    if marks >= 90:
        return 'A', 'Excellent performance!'
    elif marks >= 75:
        return 'B', 'Good job! Keep improving.'
    elif marks >= 60:
        return 'C', 'Satisfactory, but there's room for improvement.'
    else:
        return 'F', 'Needs more effort. Don't give up!'

def main():
    students = []
    print("\n Student Grade Calculator")
    while True:
        try:
            name = input("Enter student name (or type 'exit' to stop): ").strip()
            if name.lower() == 'exit':
                break
            marks = float(input("Enter marks (0-100): "))
            # Validate marks range
            if marks < 0 or marks > 100:
                print("⚠ Invalid marks! Please enter a value between 0 and 100.")
                continue
            grade, feedback = calculate_grade(marks)
            students.append((name, marks, grade, feedback))
            print(f"{name} scored {marks} and received grade {grade}.")
            print(f"Feedback: {feedback}")

        except ValueError:
            print("❌ Invalid input! Please enter numeric marks only.")

    # Display summary after all inputs
    if students:
        print("\n 📊 Summary of Results:")
        for i, (name, marks, grade, feedback) in enumerate(students, start=1):
            print(f"{i}. {name} - Marks: {marks}, Grade: {grade}, Feedback: {feedback}")
    else:
        print("\n 🚫 No student data entered.")
    if __name__ == "__main__":
        main()
```

Observation:

- ➔ This Python code creates a Flask web service with a /login route that securely authenticates users. It utilizes bcrypt for password hashing and sqlite3 for database interaction, crucially preventing SQL injection with parameterized queries.